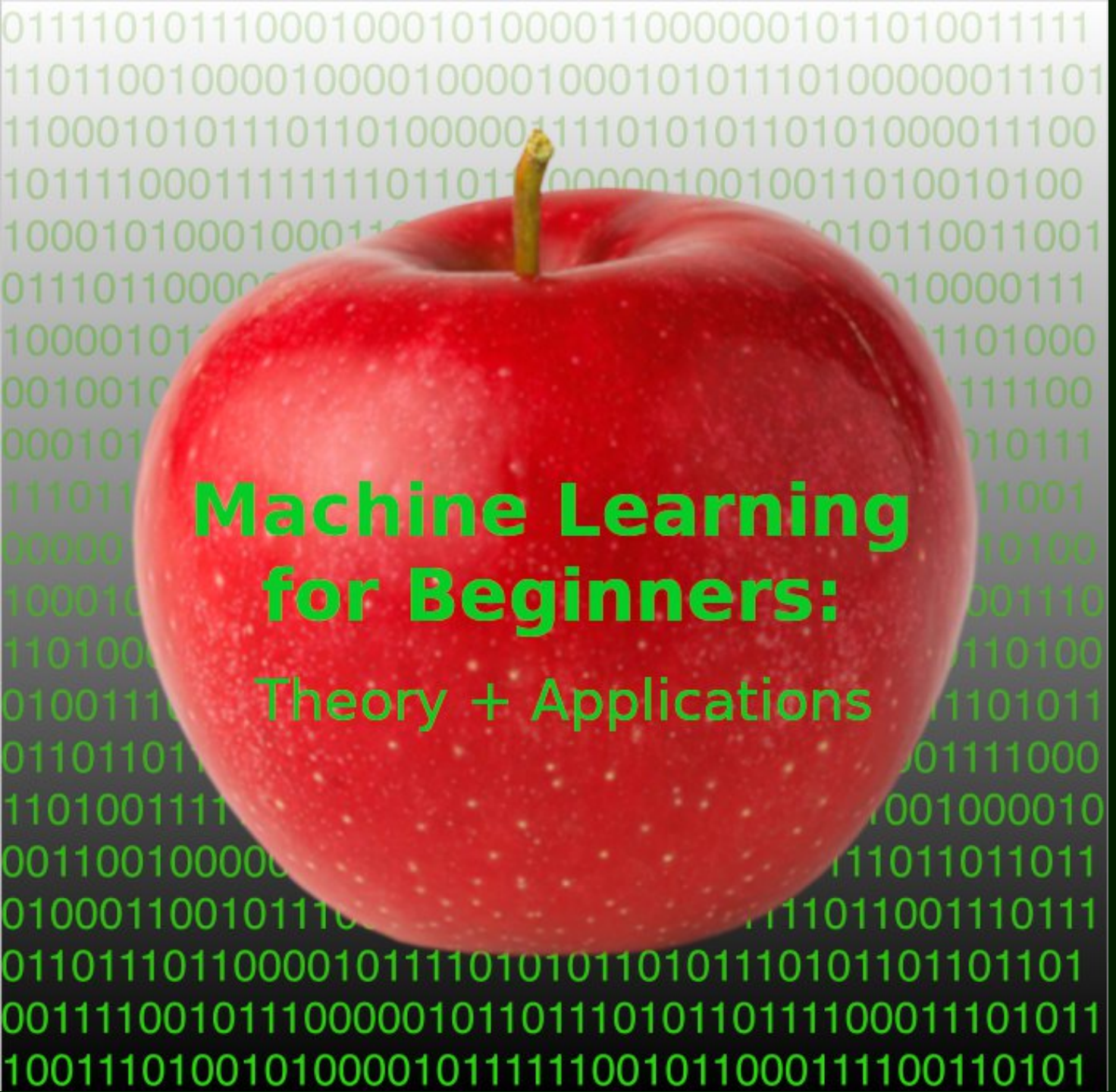


University of Bayreuth
Chair for Polymer Engineering
Prof. Dr. Holger Ruckdäschel

Title:

"Machine Learning for Beginners: Theory and Applications"

(Available in E-learning/Moodle)

Lecturer:
Dr. Rodrigo Q. Albuquerque

Lecture 2

Topics

Week 1: Introduction
ML types, data, ethics, case studies

Week 2: Preprocessing & *k*NN
Preprocessing, *k*-nearest neighbors, distance metrics, hyperparameters, cross-validation

Week 3: Linear Regression
Types, loss function, (over/under)fitting bias/variance, L1/L2 regularization, metrics

Week 4: Classification
Logistic regression, SVM, classification evaluation metrics, confusion matrix one-hot/label encoding

Week 5: Random forests
Decision trees, random forest, classification and regression applications

Week 6: PCA / KMeans
Dimensionality reduction, clustering

Week 7: Extra exercises

Week 8: Simple ANNs
Feedforward networks, activation functions, basic training

Week 9: Gaussian Processes
Kernels, gaussian processes

Week 10: Design of Experiments
DoE, random design, active learning, bayesian optimization

Weeks 11-12: Student projects

Preprocessing

Features should have similar magnitudes

Exp #	T (K)	P (Bar)	Speed (km•s^{-1})
1	497	1.50	0.00054
2	512	1.47	0.00043
3	468	1.36	0.00032

Performance
21
99
34

Preprocess features & target
(e.g., standardize)

Exp #	T‘	P‘	Speed‘
1	0.26	0.94	1.22
2	1.08	0.44	0.00
3	-1.33	-1.38	-1.22

Performance
-0.89
1.40
-0.51

Standardization (zero mean, unit variance)

$$new\ col = \frac{old\ col\ - average(old\ col)}{std(old\ col)}$$

Better ML Model, faster training

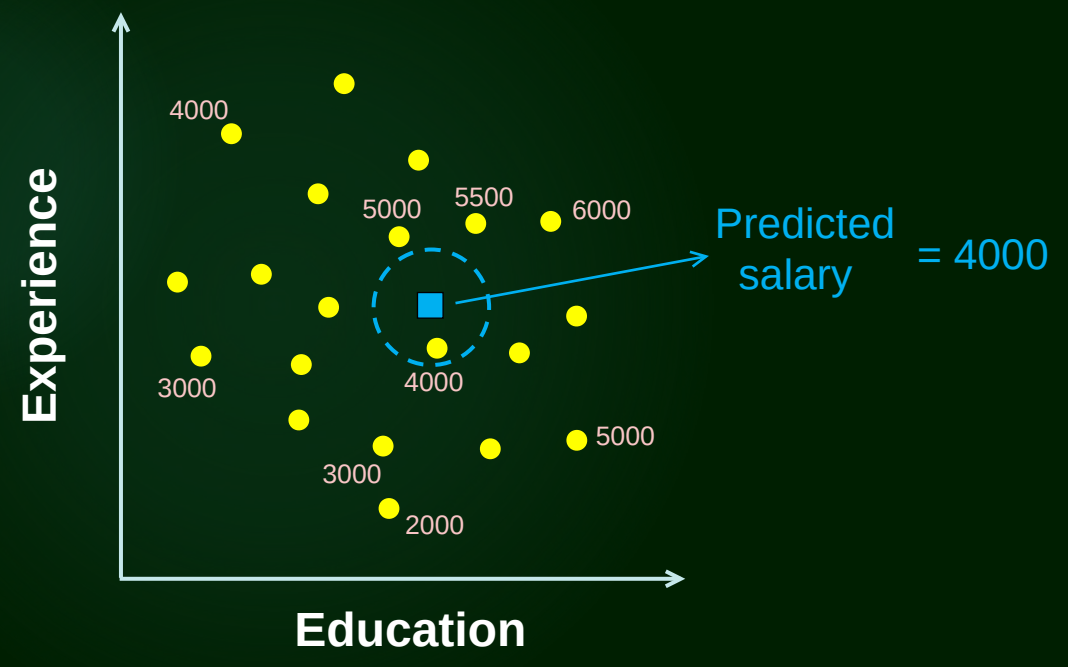
(Afterwards you can back transform the units!)

k-nearest neighbors

In *k*-NN, the neighbors decide!

k-NN: Nonparametric method (no model to be trained). Memory-based.

Current dataset: yellow
Labels: salary
New sample: blue



How to predict the salary of the new employee?

1) Calculate all distances to the new sample

Euclidean distances

$$d_{new,j} = \sqrt{\underbrace{\left(x_{new,1} - x_{j,1}\right)^2}_{\text{(education)}} + \underbrace{\left(x_{new,2} - x_{j,2}\right)^2}_{\text{(experience)}} + \cdots + \left(x_{new,d} - x_{j,d}\right)^2}$$

2) Select the *k* nearest neighbors (e.g., *k* = 1)

3) Average their target properties

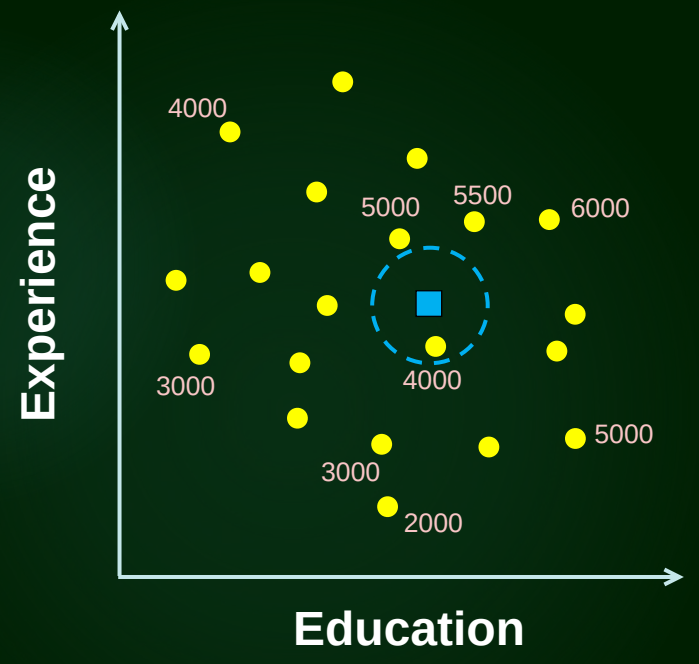
k-nearest neighbors

Influence of *k*

Note: For classification, we use *k* = 1, 3, 5, 7, etc. (odd numbers)

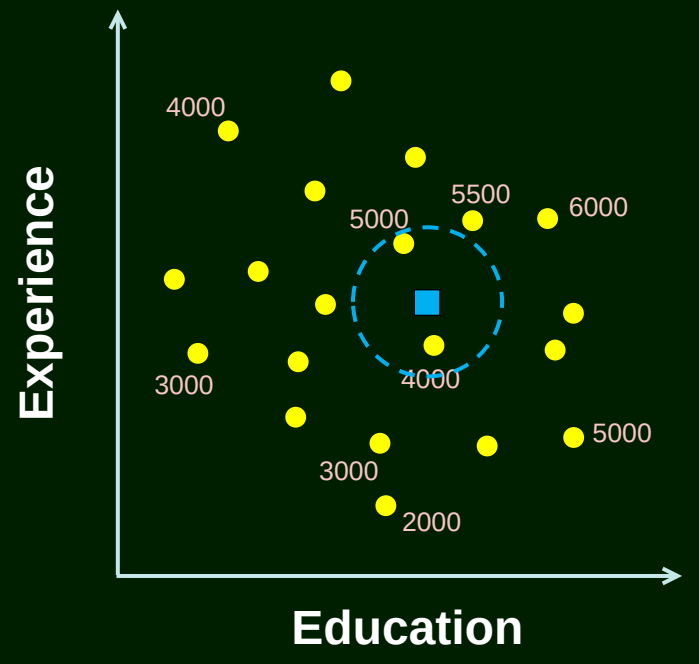
k = 1

Predicted salary = 4000



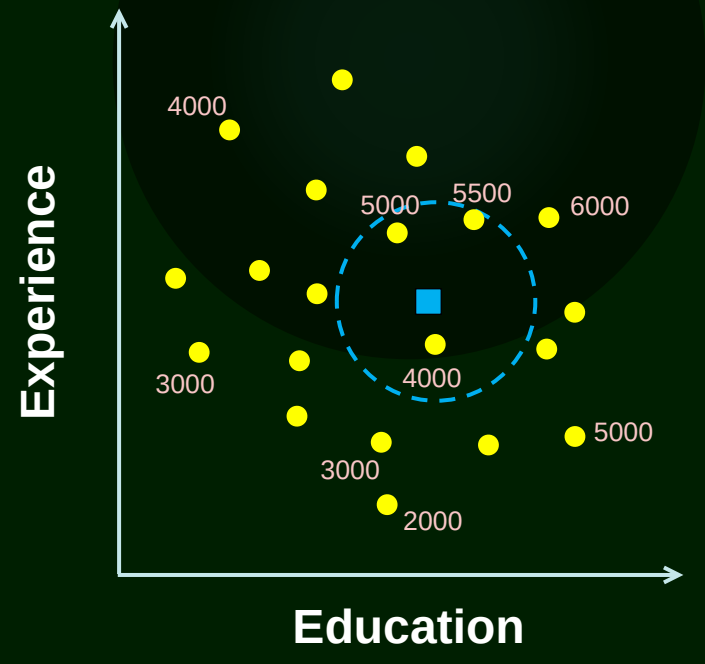
k = 2

Predicted salary = 4500



k = 3

Predicted salary = 4833

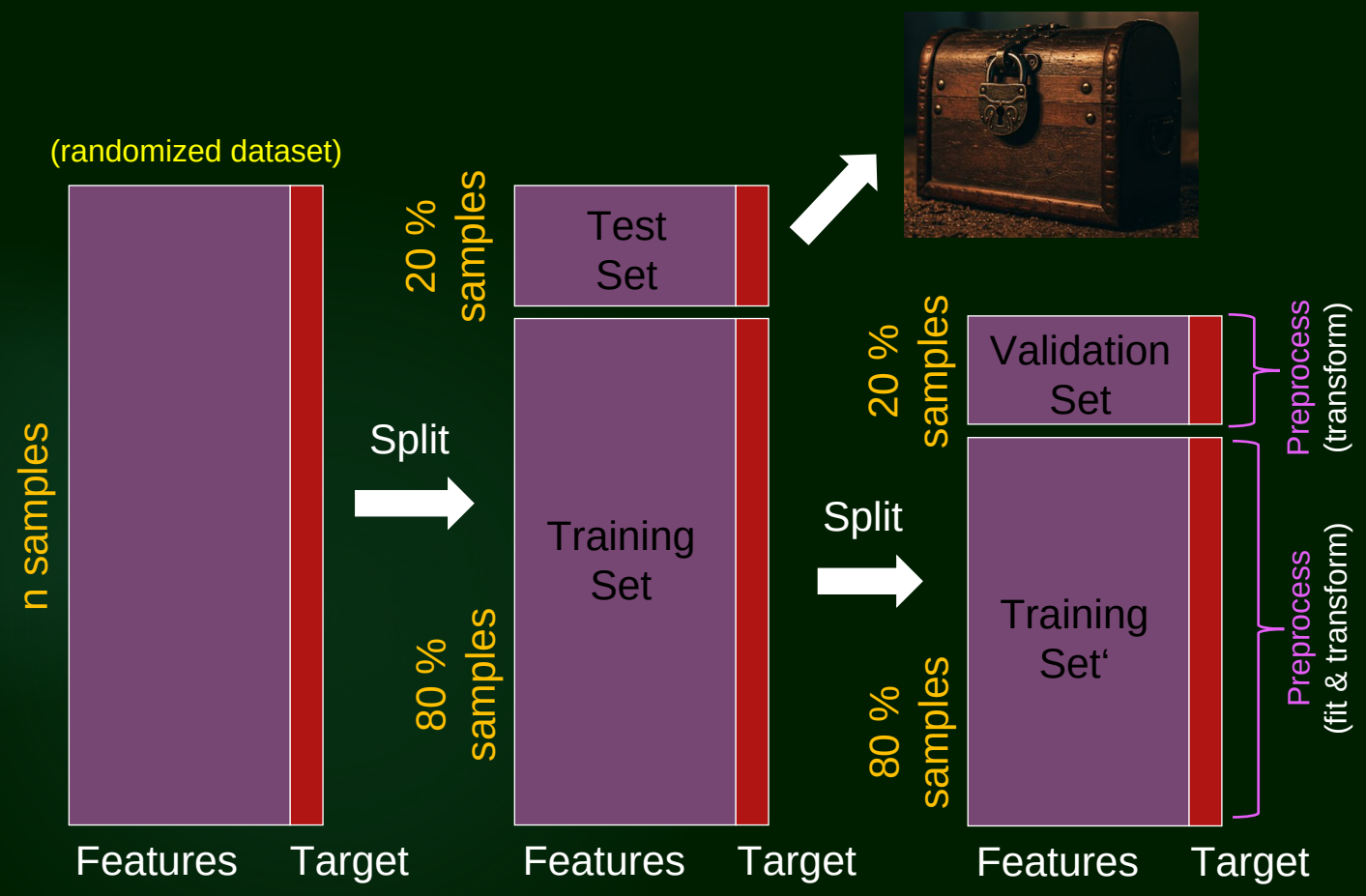


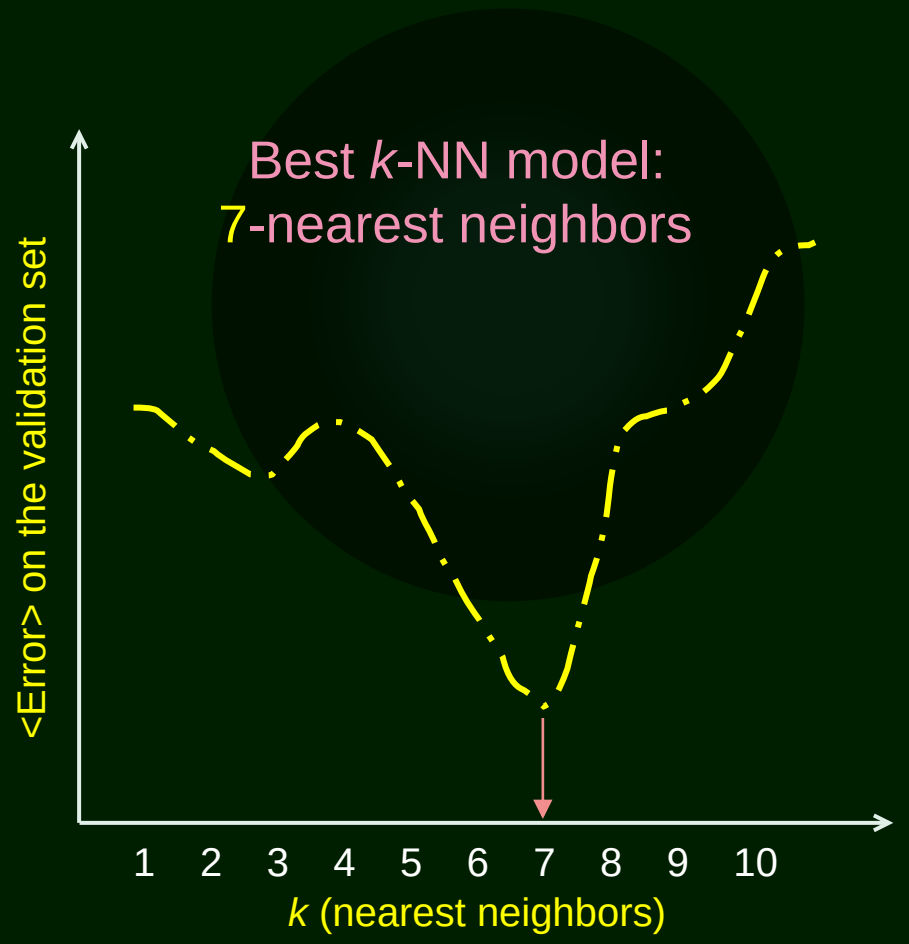
k is a hyperparameter. How to optimize it?

Hyperparameter optimization

What if the validation set was not representative?

Which hyperparameter (*k*) is better?



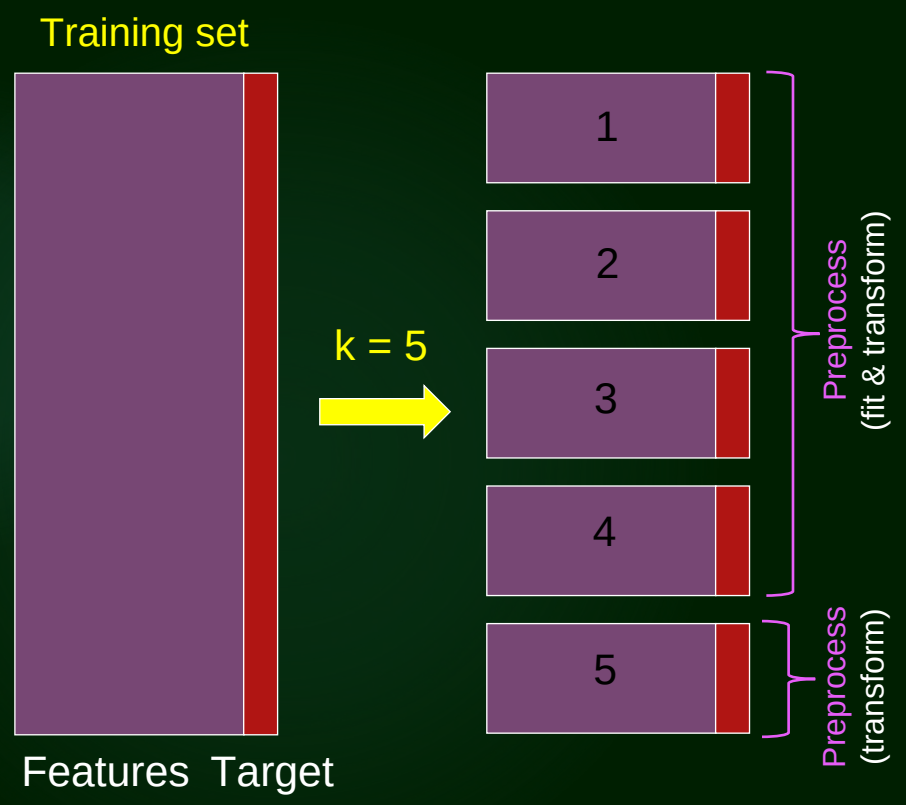


After hyperparameter optimization, you can make a FINAL evaluation using the test set

Hyperparameter optimization

k-Fold Cross-Validation (CV) => Careful, not the same k as in *k*-NN!

1) Divide the training data into k equal parts. Train the model on k - 1 parts and evaluate it on the kth part.
2) Average the performances (e.g. Errors) over all k training/validation evaluations.



Evaluation 1: Train with parts 1, 2, 3, 4. Evaluate with part 5: *Error*(1)

Evaluation 2: Train with parts 1, 2, 3, 5. Evaluate with part 4: *Error*(2)

Evaluation 3: Train with parts 1, 2, 4, 5. Evaluate with part 3: *Error*(3)

Evaluation 4: Train with parts 1, 3, 4, 5. Evaluate with part 2: *Error*(4)

Evaluation 5: Train with parts 2, 3, 4, 5. Evaluate with part 1: *Error*(5)

Overall performance = $\overline{Error} \pm std(Error)$

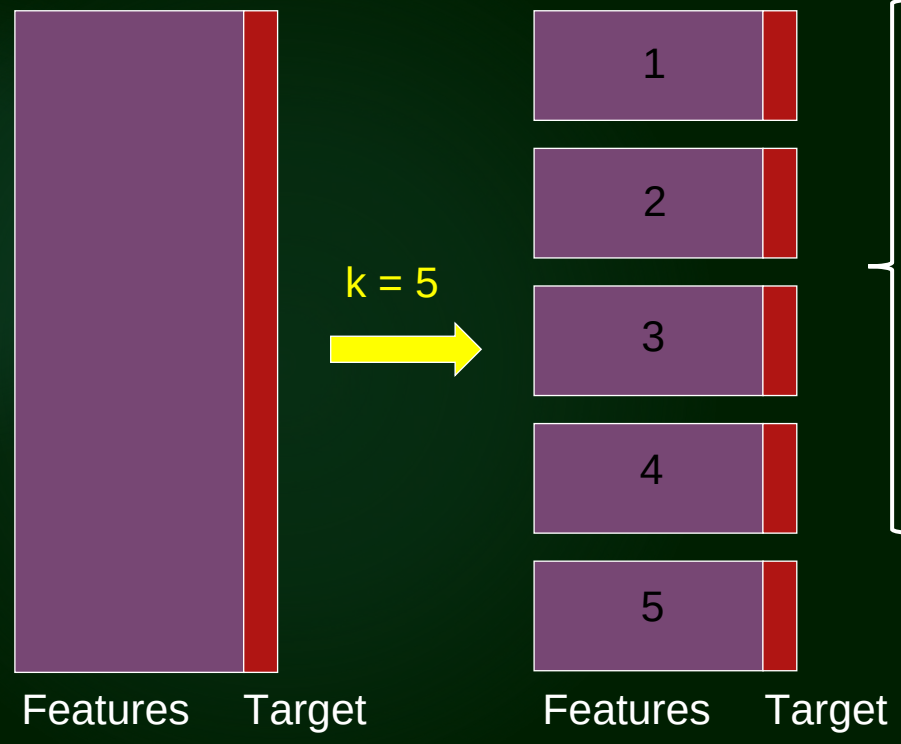
Model evaluation

Why 5-fold CV instead of e.g. 4-fold CV??

k-Fold CV => Careful, not the same k as in *k*-NN!

To evaluate a model without hyperparameter optimization, perform k-fold CV on the WHOLE dataset!

WHOLE Dataset



Evaluation 1: Train with parts 1, 2, 3, 4. Test with part 5: *Error*(1)

Evaluation 2: Train with parts 1, 2, 3, 5. Test with part 4: *Error*(2)

Evaluation 3: Train with parts 1, 2, 4, 5. Test with part 3: *Error*(3)

Evaluation 4: Train with parts 1, 3, 4, 5. Test with part 2: *Error*(4)

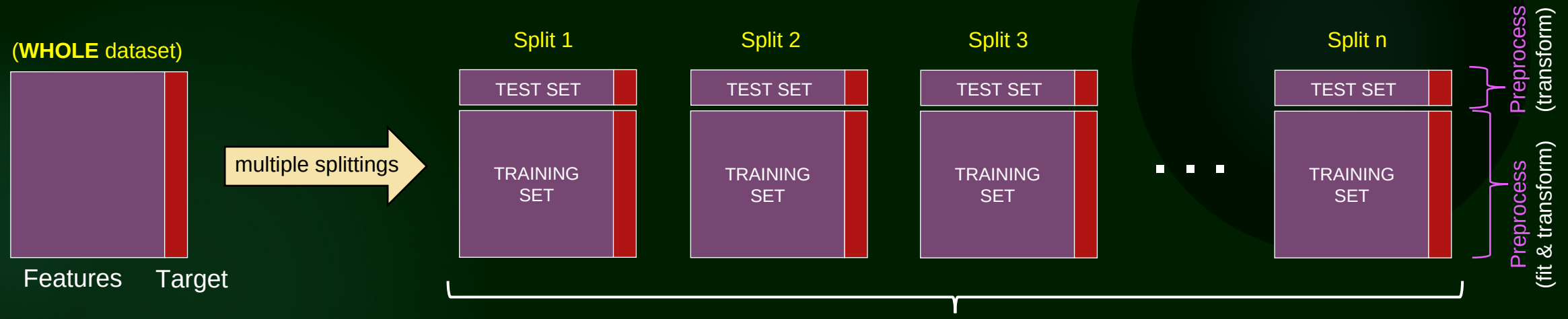
Evaluation 5: Train with parts 2, 3, 4, 5. Test with part 1: *Error*(5)

Overall performance = $\overline{Error} \pm std(Error)$

Model evaluation

Multiple random splits

Perform multiple splittings, train with training set & evaluate with test set, average results



Overall performance:

$$\overline{Error} \pm std(Error)$$

- Alternative to CV (easier to implement!)
- Less robust than CV (more sensitive to the train-test splits)
- It does not use ALL samples for training and testing (less information is used!)
- It is better than CV to estimate the model variance (for larger n)

Metrics for regression

How good is your trained model?

$$MSE = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

(Mean Squared Error)

Penalizes more the outliers

$$MAE = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i|$$

(Mean Absolute Error)

- Less sensitive to outliers
- Same unit as y (interpretability)

$$RMSE = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}$$

(Root Mean Squared Error)

- Also penalizes more the outliers, but less severely than MSE
- Same unit as y (interpretability)

$$R^2 = 1 - \frac{\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}{\sum_{i=1}^{n}(y_i - \bar{y}_i)^2}$$

(Coefficient of Determination)

$$MAPE = \frac{1}{n}\sum_{i=1}^{n}\left|\frac{y_i - \hat{y}_i}{y_i}\right| \cdot 100\%$$

(Mean Absolute Percentual Error)

- Easy (interpretability)
- VERY sensitive to small true values

➡ Always report metrics (at least) for the test set

Exercises

1) Download the file ‘files2.zip’ from moodle/e-learning (folder “Week 2”) and unzip it locally on your computer
2) Go to google colab (online) **or** launch jupyter notebook locally (offline)
3) Open/upload ‘exercise2.ipynb’ and wait for more instructions